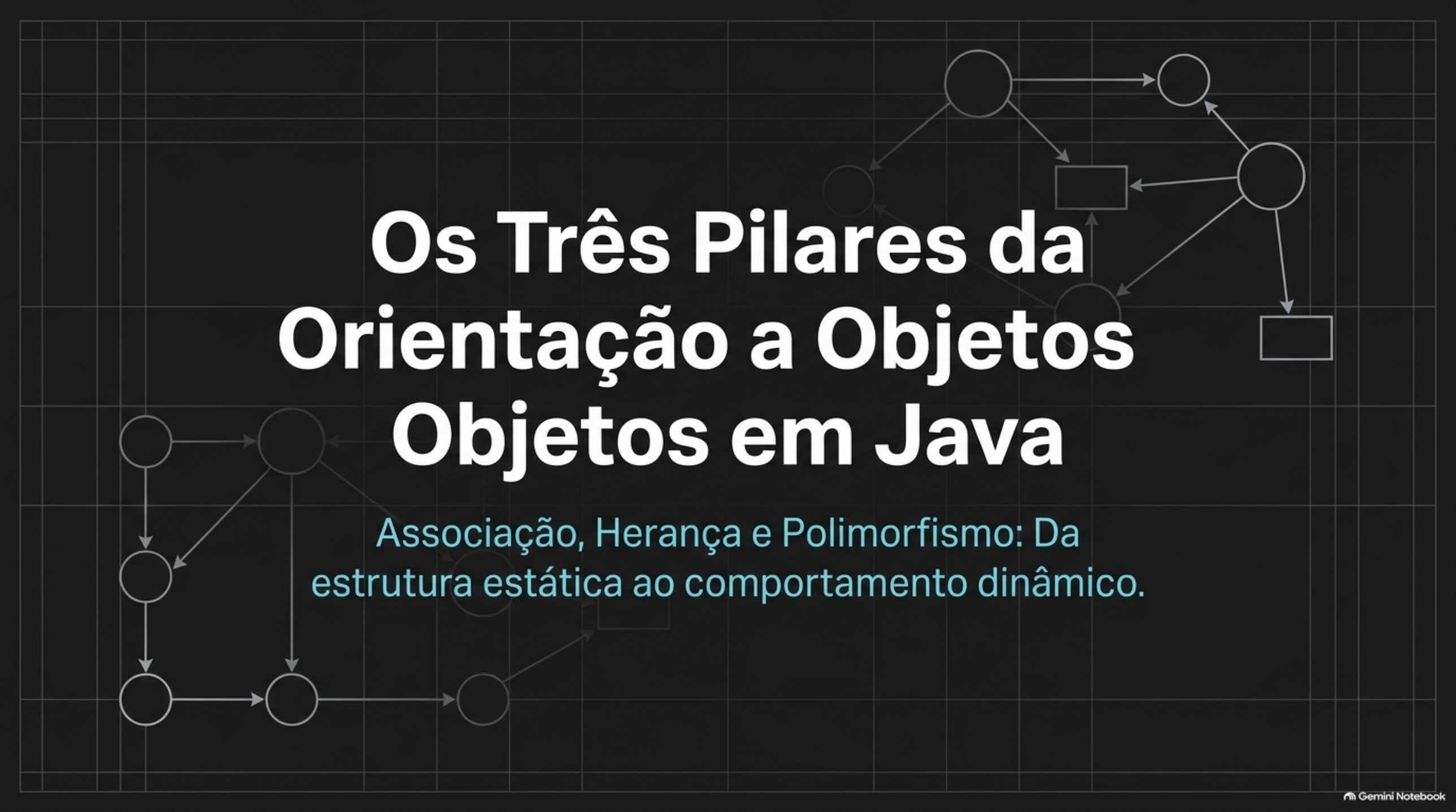


Os Três Pilares da Orientação a Objetos Objetos em Java

Associação, Herança e Polimorfismo: Da estrutura estática ao comportamento dinâmico.

Gemini Notebook

1. Conectar (Associação e Composição)

Como os objetos colaboram horizontalmente.



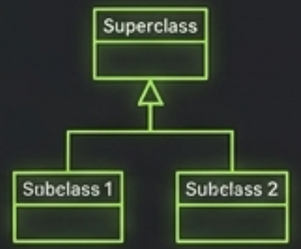

```
class A {
  B b;
}
```

referência horizontal

hierarquia vertical

2. Evoluir (Herança)

Como os objetos reaproveitam código e formam hierarquias verticais.




```
class Subclass extends Base {
  ...
}
```

referência horizontal

colaboração

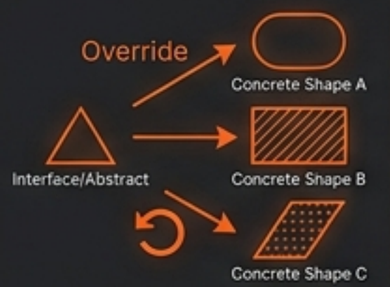
reaproveitamento

extensão

adaptação dinâmica

3. Adaptar (Polimorfismo)

Como os objetos mudam de forma e comportamento em tempo de execução.



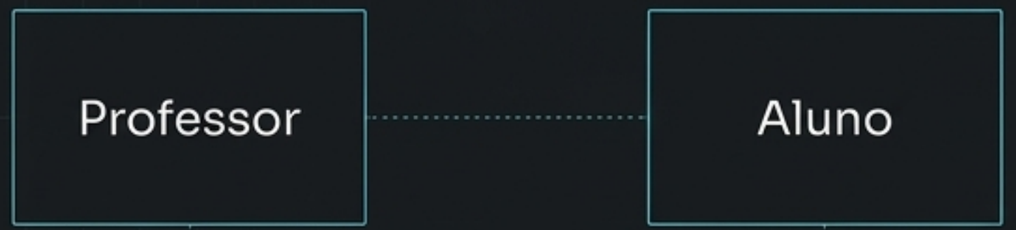

```
shape.draw()
// dynamic behavior
```

override

comportamento

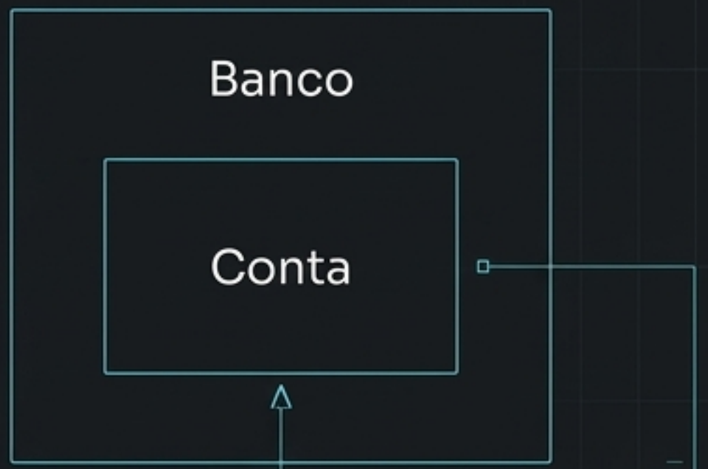
Gemini Notebook

Associação - O Relacionamento Fraco



Os objetos colaboram trocando mensagens, mas existem de forma independente. A destruição de um não afeta o outro.

Composição - O Relacionamento Forte



Relação de "Todo-Parte".

O ciclo de vida da parte depende inteiramente do todo. Se o todo é destruído, a parte também é.

Gemini Notebook

```java
class Cliente {
    String nome;
}

class Conta {
    int numero;
    Cliente titular;
}

Conta minhaConta = new Conta();
minhaConta.titular = new Cliente();
minhaConta.titular.nome = "Duke";
```

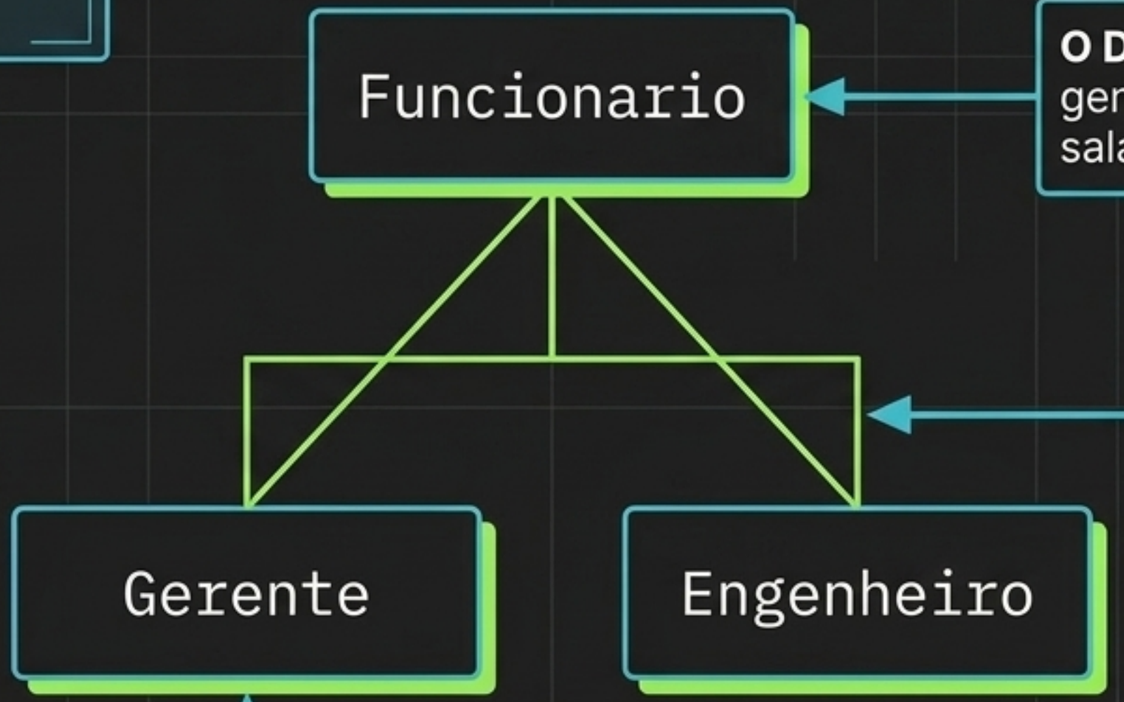
A ponte de conexão: A Conta agora *tem um* Cliente.

Navegação estrutural:

O operador ponto (.) delega a responsabilidade entre os objetos colaboradores.

Gemini Notebook

A relação ‘É-Um’ (extends)



O DNA comum: Atributos genéricos como nome e salario.

Herança estrita:
A subclasse herda automaticamente todos os atributos e métodos visíveis da superclasse.

Especialização: O Gerente *é um* Funcionário, mas adiciona comportamentos únicos (ex: autentica(senha)).

Gemini Notebook

A Classe Mãe

```
class Funcionario {
  protected double salario;

  public double getBonificacao() {
    return this.salario * 0.10;
  }
}
```

A Classe Filha

```
class Gerente extends Funcionario {

  @Override
  public double getBonificacao() {
    return super.getBonificacao() + 1000.0;
  }
}
```

Garantia do Compilador: Assegura que o contrato da classe mãe está sendo reescrito corretamente.

Reuso Inteligente: Amplia o comportamento original em vez de apenas substituí-lo.

Gemini Notebook

O tipo da referência decide o que você *pode* chamar.
O tipo do objeto decide *como* isso será executado.



`Funcionario f`
(A variável de referência)

`f = new Gerente();`

`f = new Engenheiro();`





(A **instância real** na memória)

Ligação Dinâmica (Late Binding): A JVM ignora o tipo da variável e invoca o método sobrescrito do objeto real no momento da execução.

Gemini Notebook

```java
public class ControleDeBonificacoes {
    private double totalDeBonus = 0;

    // O método aceita QUALQUER tipo de funcionário!
    public void registra(Funcionario f) {
        this.totalDeBonus += f.getBonificacao();
    }
}
```

Graças ao Polimorfismo, esta referência aceita objetos Gerente, Diretor, ou Engenheiro.

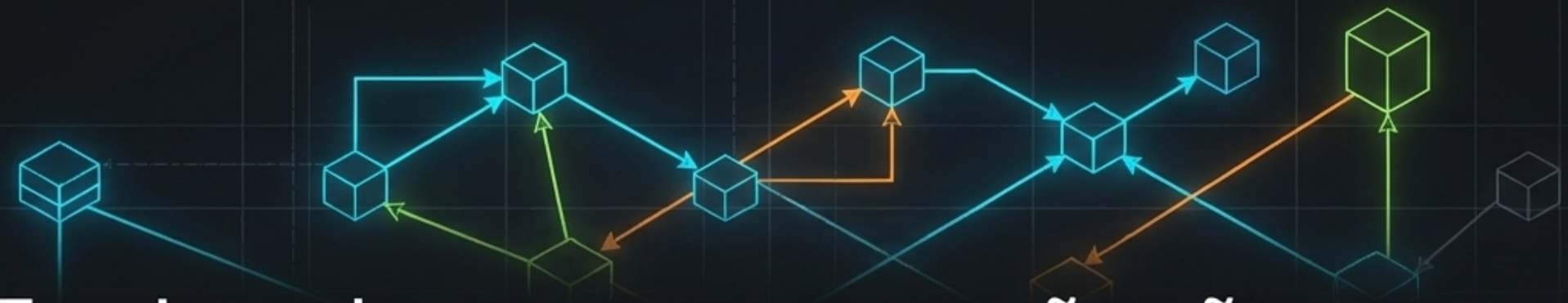
A Mágica: A JVM sabe perfeitamente qual cálculo de bônus aplicar para cada objeto.

Baixo Acoplamento: Amanhã podemos criar a classe Secretaria, e este código continuará funcionando sem nenhuma alteração!

Gemini Notebook

Conceito	Relacionamento	Mecanismo Java	Benefício Principal
Composição	Tem-Um (Todo/Parte)	Referência a outro Objeto	Delegação de responsabilidade
Herança	É-Um (Generalização)	`extends`, `super`	Reuso estrutural de código
Polimorfismo	Age-Como (Contrato)	`@Override`, `Late Binding`	Baixo acoplamento e flexibilidade extrema

Gemini Notebook



“Em Java, bons programas não são apenas coleções de variáveis e instruções.”

Eles são sociedades de objetos que colaboram (Composição), compartilham seu DNA (Herança) e se adaptam perfeitamente às regras em tempo de execução (Polimorfismo).



PROJETANDO SOFTWARE RESILIENTE, ESCALÁVEL E ELEGANTE.

Gemini Notebook